

PumpScript

Design Specification Document – Part 2

CSE 341 – Concepts of Programming Languages

Gebze Technical University – 22 May 2026

Student Name: Yusuf Eren Nalbant

Student No: 220104004049

Language: PumpScript – A DSL for Workout Routines

Deliverable: D1 – Complete Design Specification (Part 2)

This final specification is aligned with the submitted Part 2 implementation and test report. It describes the implemented subset only: no arrays, no assignment, no field access, no return values, and no real fitness tracking beyond symbolic workout output.

Contents

1	4.1 Language Overview	3
1.1	What is PumpScript?	3
1.2	Sample Program	3
1.3	Design Goal	3
2	4.2 Lexical Structure	3
3	4.3 Syntax	4
3.1	Complete EBNF Grammar	4
3.2	Ambiguity Resolution	4
4	4.4 Semantics [Part 2]	5
4.1	Construct 1: <code>repeat N times { B }</code>	5
4.2	Construct 2: <code>exercise Block</code>	5
5	4.5 Type System [Part 2]	5
5.1	Primitive Types and Values	5
5.2	Strong Typing	5
5.3	Implicit Coercion	6
5.4	Type Equivalence	6
5.5	Structured Type: Exercise Record	6
6	4.6 Names, Binding, Scope, and Lifetime	6
6.1	Legal Identifiers	6
6.2	Compile-time vs Run-time Binding	6
6.3	Static Scoping	6
6.4	Lifetime	7
7	4.7 Expressions and Assignment [Part 2]	7
7.1	Precedence and Associativity	7
7.2	Boolean Operators	7
7.3	Operand Evaluation Order	7
7.4	Assignment	7
8	4.8 Design Rationale [Part 2]	7
8.1	Decision 1: Strong Typing	7
8.2	Decision 2: Only <code>int -> float</code> Coercion	7
8.3	Decision 3: Name Equivalence for Exercise Records	7
8.4	Decision 4: Omit Boolean Connectives	8
8.5	Decision 5: No Assignment	8

1 4.1 Language Overview

1.1 What is PumpScript?

PumpScript is a domain-specific language for defining and executing workout routines. Its target users are athletes, fitness coaches, and students who want to describe training sessions using concepts from the fitness domain instead of writing a general-purpose script. A PumpScript program can define parameterized **routines**, create typed **exercise** records, print rest periods with **rest N seconds**, and control execution with **if** and **repeat**.

The DSL prioritizes Sebesta's **readability** and **reliability**. Readability is improved because the main statements look like workout instructions. Reliability is improved through static type checking: fields such as **sets**, **reps**, **weight**, **group**, and **completed** have fixed expected types. PumpScript knowingly sacrifices generality and some writability: there are no arrays, assignment statements, general variables, or first-class record values. This keeps the language small enough to explain and defend.

1.2 Sample Program

The following complete program is valid under the final implementation. It shows a routine, an exercise block, a rest statement, a top-level conditional, and a repeat statement.

```
routine chest_day(sets: int) {  
  exercise bench_press {  
    sets: sets  
    reps: 10  
    weight: 80.0  
    group: "chest"  
    completed: false  
  }  
  rest 60 seconds  
}  
  
if true {  
  chest_day(4)  
} else {  
  chest_day(3)  
}  
  
repeat 1 times {  
  rest 30 seconds  
}
```

1.3 Design Goal

The key design goal is not to build a full programming language. The goal is to make the domain actions explicit and safe. Instead of letting users write arbitrary dictionaries or print statements, PumpScript gives them domain constructs: **exercise** for structured workout data and **rest** for recovery periods. This makes the language a genuine DSL rather than a themed generic toy language.

2 4.2 Lexical Structure

The lexer is hand-written. It applies maximal munch for numbers and two-character operators, skips whitespace, and supports `//` single-line comments. Tokens carry line numbers for error messages.

Category	Pattern / Values	Example
Identifier	[A-Za-z_] [A-Za-z0-9_]* excluding keywords	bench_press, sets
Integer literal	[0-9]+	4, 90
Float literal	[0-9]+. [0-9]+	80.0, 2.5
String literal	double-quoted text, no newline escapes	"chest"
Boolean literal	true, false	false
Keywords	routine exercise rest repeat times if else seconds int float string bool true false	routine
Operators	+ - * / > < >= <= == !=	sets + 1
Separators	{ } () : ,	block and parameter syntax
Comments	// to end of line	// chest day

There is no semicolon token in the final language. This was intentional: blocks and line-independent statement forms are enough for the small DSL, and removing semicolons improves readability for non-programmers.

3 4.3 Syntax

3.1 Complete EBNF Grammar

The grammar below uses Sebesta-style EBNF: braces mean repetition, brackets mean optional parts, and alternatives are separated by |. The implementation is a recursive-descent parser with one method for each major nonterminal.

```

<program> = { <routine_decl> | <statement> } EOF

<routine_decl> = "routine" IDENT "(" [ <param_list> ] ")" <block>
<param_list> = <param> { "," <param> }
<param> = IDENT ":" <type>
<type> = "int" | "float" | "string" | "bool"

<block> = "{" { <statement> } "}"
<statement> = <exercise_block>
               | <rest_stmt>
               | <repeat_stmt>
               | <if_stmt>
               | <routine_call>

<exercise_block>= "exercise" IDENT "{" { <field> } "}"
<field> = IDENT ":" <expr>
<rest_stmt> = "rest" INT_LIT "seconds"
<repeat_stmt> = "repeat" INT_LIT "times" <block>
<if_stmt> = "if" <expr> <block> [ "else" <block> ]
<routine_call> = IDENT "(" [ <arg_list> ] ")"
<arg_list> = <expr> { "," <expr> }

<expr> = <comparison> { ("==" | "!=") <comparison> }
<comparison> = <term> { (">" | "<" | ">=" | "<=") <term> }
<term> = <factor> { ("+" | "-") <factor> }
<factor> = <unary> { ("*" | "/" ) <unary> }
<unary> = "-" <primary> | <primary>
<primary> = INT_LIT | FLOAT_LIT | STRING_LIT | BOOL_LIT
            | IDENT | "(" <expr> ")"

```

3.2 Ambiguity Resolution

The grammar resolves expression ambiguity by splitting expressions into precedence levels. Unary minus binds most tightly, then multiplication/division, addition/subtraction, relational comparison, and finally equality. All binary operators are left-associative because the parser consumes repeated operators in loops. The dangling-else ambiguity is also resolved by the parser: an **else** belongs to the nearest preceding unmatched **if**, since it is parsed immediately after the then-block.

4 4.4 Semantics [Part 2]

I use big-step operational semantics because the interpreter executes whole statements and produces output. Let env be the current environment, out the output stream, and $\langle S, env, out \rangle \Downarrow \langle env', out' \rangle$ mean that statement S executes from env, out to env', out' .

4.1 Construct 1: repeat N times { B }

The count is an integer literal in the grammar, so it is fixed before the loop body runs. A zero-count repeat leaves the state unchanged:

$$\overline{\langle \text{repeat } 0 \text{ times } B, env, out \rangle \Downarrow \langle env, out \rangle}$$

For a positive count, PumpScript executes the body once and then repeats the remaining $N - 1$ iterations:

$$\frac{\langle B, env, out \rangle \Downarrow \langle env_1, out_1 \rangle \quad \langle \text{repeat } (N - 1) \text{ times } B, env_1, out_1 \rangle \Downarrow \langle env_2, out_2 \rangle}{\langle \text{repeat } N \text{ times } B, env, out \rangle \Downarrow \langle env_2, out_2 \rangle} \quad N > 0$$

The trade-off is simplicity over expressiveness: **repeat** cannot use a changing expression as its count, but it is predictable and easy to type-check.

4.2 Construct 2: exercise Block

An exercise block evaluates every field expression in the current environment and appends formatted workout output. The environment is not extended with field names; fields are local to the printed exercise record.

$$\frac{eval(e_1, env) = v_1 \quad \dots \quad eval(e_k, env) = v_k \quad fieldsOK(name_i, v_i)}{\langle \text{exercise } E \{name_1 : e_1, \dots, name_k : e_k\}, env, out \rangle \Downarrow \langle env, out \cdot format(E, fields) \rangle}$$

The type checker enforces the fixed exercise schema before execution. At runtime, domain-specific safety checks reject non-positive **sets/ reps** and negative **weight**. This separation is intentional: static errors catch type mistakes, while runtime errors catch invalid values that are still type-correct.

5 4.5 Type System [Part 2]

5.1 Primitive Types and Values

PumpScript has four primitive types:

Type	Value set	Typical use
int	whole numbers	sets, reps, rest seconds, repeat count
float	decimal numbers	weight in kilograms
string	text literals	muscle group names such as " chest "
bool	true, false	completion status and conditions

5.2 Strong Typing

PumpScript is strongly typed. A static type checker verifies routine calls, parameter types, exercise field types, condition types, and expression operators before interpretation. For example, **weight: "heavy"** is rejected before any exercise output is produced. Undefined identifiers are also rejected by the type checker.

5.3 Implicit Coercion

The only implicit coercion is widening from `int` to `float`. It is allowed when a `float` is expected and an `int` is provided, and in mixed numeric arithmetic. No narrowing from `float` to `int` is allowed. There is no coercion between numeric values and strings or booleans.

Arithmetic rules are: `int op int -> int` for `+`, `-`, and `*`; if either operand is `float`, the result is `float`; division `/` always returns `float`. Relational operators require numeric operands and return `bool`. Equality works only for compatible primitive types, including numeric `int/float` comparison by widening.

5.4 Type Equivalence

For the structured exercise type, PumpScript chooses name equivalence rather than structural equivalence. The record identity is tied to the exercise construct and the exercise name, not merely to a matching list of fields. In the current implementation, exercise records are not first-class values passed between routines. Therefore the practical enforcement is: every `exercise` block must follow the exact declared exercise field schema, and arbitrary structurally similar records are not supported. This keeps the design easy to defend: the language has one named structured record form, not anonymous dictionary-like records.

5.5 Structured Type: Exercise Record

The structured type is an `exercise` record with this fixed schema:

Field	Type	Meaning
<code>sets</code>	<code>int</code>	number of sets; must be positive at runtime
<code>reps</code>	<code>int</code>	repetitions per set; must be positive at runtime
<code>weight</code>	<code>float</code>	kilograms; must be non-negative at runtime
<code>group</code>	<code>string</code>	muscle group label
<code>completed</code>	<code>bool</code>	whether the exercise is complete

The final type checker rejects unknown fields, duplicate fields, missing required fields, and wrong field types. This decision favors reliability over flexibility.

6 4.6 Names, Binding, Scope, and Lifetime

6.1 Legal Identifiers

Identifiers match `[A-Za-z_][A-Za-z0-9_]*` and must not be reserved keywords. Examples of legal identifiers are `chest_day`, `sets`, and `bench_press`. Examples of illegal identifiers are `3sets` and `routine`.

6.2 Compile-time vs Run-time Binding

Lexical classes, parse structure, routine signatures, parameter types, exercise field schemas, and expression result types are bound before execution. Routine declarations are collected in a first pass so calls may be checked and resolved. Runtime binding occurs for actual argument values, parameter values inside each routine call, evaluated exercise field values, and output production.

6.3 Static Scoping

PumpScript uses static scoping. A routine call creates a new local environment whose parent is the global environment, not the caller's local environment. This means a routine can use its own parameters and globally registered routines, but it cannot accidentally capture a caller's temporary parameter. If the language used dynamic scoping, the meaning of a routine could change depending on who called it, which would reduce reliability.

6.4 Lifetime

Routine declarations have static lifetime for the duration of program execution. Routine parameters have stack-dynamic lifetime: they are created when a routine call begins and disappear when the call returns. Exercise records are temporary output objects; their fields are evaluated and printed, not stored as heap objects.

7 4.7 Expressions and Assignment [Part 2]

7.1 Precedence and Associativity

Level	Operators	Associativity
1 highest	unary -	right/one operand
2	* /	left
3	+ -	left
4	> < >= <=	left
5 lowest	== !=	left

7.2 Boolean Operators

The final implementation does not include `and`, `or`, `&&`, or `||`. Therefore there is no short-circuit boolean connective to define. Conditions are still boolean expressions, such as `true`, `sets > 3`, or `weight == 80.0`.

7.3 Operand Evaluation Order

Operand evaluation is defined as left-to-right. This matches the tree-walking interpreter and improves reliability because expressions behave predictably.

7.4 Assignment

There is no assignment statement and assignment is not an expression. Values enter the language through literals, routine parameters, and exercise field expressions. This sacrifice makes the interpreter simpler and avoids questions about variable mutation and side effects.

8 4.8 Design Rationale [Part 2]

8.1 Decision 1: Strong Typing

Strong typing was chosen because fitness data has clear categories. A string weight or a boolean repetition count is almost certainly an error. The trade-off is that the language rejects some informal notation, but it gives better reliability and clearer error messages.

8.2 Decision 2: Only `int` $\rightarrow$ `float` Coercion

The single widening coercion improves writability without hiding dangerous mistakes. A user may naturally write `weight: 80` instead of `80.0`; accepting this is harmless. The opposite direction, `float` $\rightarrow$ `int`, could silently lose information, so it is rejected.

8.3 Decision 3: Name Equivalence for Exercise Records

Name equivalence fits the domain because exercise identity matters. A squat and a bench press may have the same field structure, but they are not the same workout movement. Structural equivalence would make the language more flexible, but it could accept semantically wrong substitutions. The accepted trade-off is less generic reuse in exchange for safer domain meaning.

8.4 Decision 4: Omit Boolean Connectives

Boolean connectives were omitted to keep the final implementation small and testable. Users can still write useful conditions using boolean literals, equality, and numeric comparisons. The trade-off is reduced expressiveness; complex conditions must be simplified or split across statements in a future version.

8.5 Decision 5: No Assignment

Assignment was omitted because PumpScript is meant to describe routines, not compute changing program state. Removing assignment also removes many binding and lifetime complications. The cost is that users cannot build counters or accumulators, but the benefit is a much easier language to reason about and defend in the exam.